Ficha 7 (26) - Exercícios de Views e Templates

Ficha 7 - Exercícios de Views e Templates

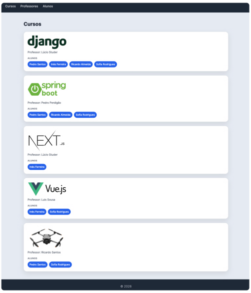
Nesta ficha irá continuar a explorar o Django, criando views e templates para as aplicações que criou.

1. Irá primeiro implementar uma aplicação de raiz, para uma escola.
2. Depois, irá criar views para o seu protfólio.

Exemplo de Implementação

> ⓘ Vamos implementar, de forma guiada, uma aplicação Django baseada no padrão MVT para compreender os passos. Leia e siga os passos de implementação quando indicado.

Vamos implementar uma plataforma online que apresenta os cursos de uma escola. Cada curso é lecionado por um único professor, embora cada professor possa lecionar em vários cursos. Cada curso tem um nome e uma imagem. Pode ter vários alunos, e cada aluno pode frequentar vários cursos. Cada aluno tem um nome e número.



Cursos

Professor: Lúcio Studer
ALUNOS
Pedro Santos · Inês Ferreira · Ricardo Almeida · Sofia Rodrigues

Professor: Pedro Perdigão
ALUNOS
Pedro Santos · Ricardo Almeida · Sofia Rodrigues

Professor: Lúcio Studer
ALUNOS
Inês Ferreira

Professor: Luís Sousa
ALUNOS
Inês Ferreira · Sofia Rodrigues

Professor: Ricardo Santos
ALUNOS
Pedro Santos · Sofia Rodrigues

Exemplo de aplicação a correr: https://escola.pw.deisi.ulusofona.pt/ ↗

Modelação

A primeira coisa a fazer para qualquer nova aplicação é a sua modelação. É o que faremos no exemplo em baixo.

1.1 Vamos identificar as entidades, relações e diagrama de entidades e relações:

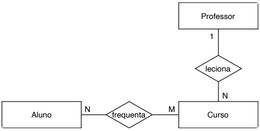
A. **Entidades e atributos**

- Curso: id do curso, nome, imagem;
- Professor: id do professor, nome, email;
- Aluno: id do aluno, número, nome.

B. **Relacionamentos**

- *Leciona:*
    - Entre Professor – Curso
    - Tipo: 1:N
        - Um professor pode lecionar vários cursos
        - Um curso é lecionado por um único professor.
- *Frequenta:*
    - Entre: Aluno – Curso
    - Tipo: N:M
        - Um aluno pode frequentar vários cursos
        - Um curso pode ter vários alunos

C. **Diagrama Entidade Relação (DER)**



Esta fase conceptual é chave. Agora é só implementar.

Implementação da Modelação

Vamos agora implementar esta modelação. Os passos são sempre os mesmos: criar a app, registar no settings, criar os modelos, migrar e registar no site do admin.

No projeto Django em que vai implementar esta aplicação, caso tenha uma aplicação `escola`, nos passos em baixo use como nome de aplicação `escola-online` (no `startapp` e no `INSTALLED_APPS`).

1. No seu repositório *GitHub* onde tem o seu portfolio, usando o *Codespaces*, criar uma nova aplicação:

```
python manage.py startapp escola
```

2. registar a aplicação em `project/settings.py` em `INSTALLED_APPS`:
3.

```
## ficheiro project/settings.py

INSTALLED_APPS = [
    ## ...
    'escola',
]
```

4. criar os modelos em `escola/models.py`

```
## ficheiro models.py

class Professor(models.Model):
    nome = models.CharField(max_length=100)
    email = models.EmailField()

    def __str__(self):
        return self.nome

class Aluno(models.Model):
    nome = models.CharField(max_length=100)
    numero = models.CharField(max_length=20, unique=True)

    def __str__(self):
        return self.nome

class Curso(models.Model):
    nome = models.CharField(max_length=100)
    imagem = models.ImageField(upload_to='cursos/')
    professor = models.ForeignKey(Professor, on_delete=models.CASCADE, related_name='curs
    alunos = models.ManyToManyField(Aluno, related_name='cursos')

    def __str__(self):
        return self.nome
```

5. criar as migrações, que especificam as mudanças:

   ```
   python manage.py makemigrations
   ```

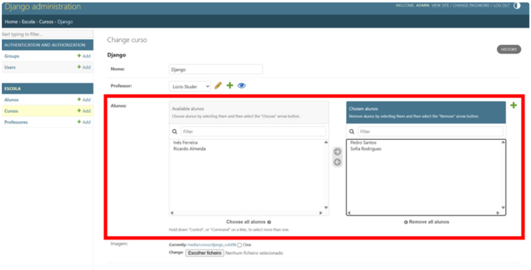
6. aplicar as migrações geradas para criar na base de dados as tabelas da aplicação

   ```
   python manage.py migrate
   ```

7. em `admin.py` registar os modelos com `admin.site.register()`

```
from django.contrib import admin
from .models import Curso, Professor, Aluno

admin.site.register(Aluno)
admin.site.register(Professor)

class CursoAdmin(admin.ModelAdmin):
    filter_horizontal = ('alunos',)  # permite adicionar mais facilmente alunos ao curso

admin.site.register(Curso, CursoAdmin)
```

> ⓘ a definiçao de `filter_horizontal = ('alunos',)` permite adicionar mais facilmente alunos ao curso, como em baixo.


Django administration
WELCOME, ADMIN. VIEW SITE / CHANGE PASSWORD / LOG OUT
Home › Escola › Cursos › Django
Start typing to filter...
AUTHENTICATION AND AUTHORIZATION
Groups + Add
Users + Add
ESCOLA
Alunos + Add
Cursos + Add
Professores + Add
HISTORY
Available alunos
Choose alunos by selecting them and then select the "Choose" arrow button.
Filter
Chosen alunos
Remove alunos by selecting them and then select the "Remove" arrow button.
Filter
Choose all alunos
Remove all alunos
Hold down "Control", or "Command" on a Mac, to select more than one.
Escolher ficheiro

Change curso

Django

Nome: Django

Professor: Lúcio Studer

Alunos:
Available alunos: Inês Ferreira, Ricardo Almeida
Chosen alunos: Pedro Santos, Sofia Rodrigues

Imagem:
Currently: media/cursos/django_ru6d9k ☐ Clear
Change: Escolher ficheiro Nenhum ficheiro selecionado

8. usando o interface de administração (na rota `/admin`) insira alguns dados em cada uma das tabelas da aplicação e valide se está tudo conforme. Pode usar como imagens dos cursos os logos.zip ↗.

> ⓘ Chegando a este ponto, consegue manipular os dados da base de dados usando a aplicação admin do Django.

Rota Base da Aplicação

Vamos agora criar uma aplicação web que permita fazer operações CRUD na base de dados através de páginas HTML, em vez do interfaces admin. A primeira coisa a fazer é definir uma rota para a aplicação `escola`.

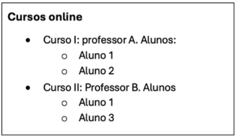
1. Na pasta `project/`, no ficheiro `urls.py` adicione à lista `urlpatterns` o caminho para cada a aplicação.:

```
## ficheiro projects/urls.py

from django.contrib import admin
from django.urls import path, include              #     <- adicionar include
from django.conf import settings
from django.conf.urls.static import static

urlpatterns = [
    path("admin/", admin.site.urls),
    path("escola/", include("escola.urls")),       #     <- rota para aplicação web escola
]

# Serve ficheiros media em DEBUG
if settings.DEBUG:
    urlpatterns += static(settings.MEDIA_URL, document_root=settings.MEDIA_ROOT)
```

Quando cria uma aplicação em Django, por omissão não está acessível ao exterior. Da mesma forma que regista as tabelas que quer que fiquem visíveis no `admin.py`, só quando define uma rota em `projects/urls.py` é que pode tornar acessíveis os seus conteúdos.

Visualização dos Cursos

Vamos criar uma página que permita visualizar da seguinte forma a informação associada aos cursos.



Cursos online

- Curso I: professor A. Alunos:
    - Aluno 1
    - Aluno 2
- Curso II: Professor B. Alunos
    - Aluno 1
    - Aluno 3

Visualização dos cursos, com seu professor e alunos

Para criar uma página web associada à aplicação `escola`, os passos são sempre os mesmos:

1. em `escola/views.py`, criar uma função view capaz de receber um pedido e retornar uma página HTML.
2. na pasta `escola/templates/escola/` criar o respetivo ficheiro HTML, que no Django denominamos de template HTML
3. em `escola/urls.py` criar um `path`, caminho/rota para a função view

Observando a imagem acima:

- Queremos iterar por que entidade?
- Iteramos pelos cursos!
- E depois, para cada curso?
- Iteramos pelos seus alunos e pelo seu professor!

Ok. Então temos que passar ao HTML a lista de `cursos`. Com esta conseguimos, para cada `curso`, aceder ao campo `alunos` que nos permitirá apresentar a lista de alunos e ao campo `professor`, que nos dará info do professor. Eis a função *view* que recolhe os dados necessários para o template apresentar esta informação como ilustrado em cima:

```
## ficheiro escola/views.py

from django.shortcuts import render
from .models import Curso

def cursos_view(request):
    cursos=Curso.objects.all()
    return render(request, 'escola/cursos.html', {'cursos': cursos})
```

Melhoria

Apresentamos em baixo uma versão mais eficiente da função `cursos_view` , que recorrev a `select_related` e `prefetch_related`.

```
##  ficheiro escola/views.py

from django.shortcuts import render
from .models import Curso

def cursos_view(request):

    cursos = Curso.objects
               .select_related('professor')
               .prefetch_related('alunos')
               .all()

    return render(request, 'escola/cursos.html', {'cursos': cursos})
```

Podemos tornar o código da função `cursos_view` mais eficiente. Depois de obter a lista de cursos (1 query), no template HTML, para cada curso será necessário aceder novamente à base de dados para obter a informação do respetivo professor (N queries) e a lista de alunos (N queries).

Ou seja, no total, teremos **1 + 2N queries**, o que é pouco eficiente em termos de desempenho.

O ideal será recolher toda esta informação de forma antecipada, evitando múltiplos acessos à base de dados.

Uma abordagem mais eficiente consiste em **carregar previamente os dados relacionados**, reduzindo significativamente o número de queries.

Para isso, podemos usar:

- `select_related('professor')` → para relações *ForeignKey*
- `prefetch_related('alunos')` → para relações *ManyToMany*

Desta forma, ao iterar sobre os cursos, já teremos disponível a informação do professor e dos alunos, sem necessidade de novas consultas à base de dados.

Estrutura de Templates

> Agora vamos criar o *template* HTML, que integrará os dados enviados pela view, e será devolvido ao browser que fez o pedido, ficando renderizado como ilustrado na página anterior

crie uma pasta `templates/escola`, e nesta crie:

- o ficheiro `base.html` que servirá de base para todos os templates
- o ficheiro `cursos.html` que renderiza os conteúdos recebidos da função cursos_view. Este ficheiro usará `base.html` como layout base.

`base.html`

Veja o código em baixo. tem o template base de todas a páginas da aplicação

> O ficheiro `base.html` serve de layout base para todos as páginas/templates da aplicação. Assim todas as paginas têm um aspecto/layout semelhante, com o mesmo cabeçalho, menu e rodapé. Apenas o conteúdo do bloco `{% block content %}{% endblock %}` é que varia de página para página. Sempre que criar uma nova página, deverá atualizar o menu.
>
> Utilize elementos HTML5 (`header`, `h1`, `nav`, `main`, `footer`) e siga boas práticas de UI/UX. Para já não se preocupe em estilizar os elementos.

```
<!-- templates/escola/base.html -->

<!DOCTYPE html>
<html lang="pt">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Escola</title>
</head>
<body>
    <header>
        <h1>Escola</h1>
        <nav>
            <a href="{% url 'cursos' %}">Cursos</a>
        </nav>
    </header>


    <main>
        {% block content %}
        <!-- Conteúdo dinâmico -->
        {% endblock %}
    </main>

    <footer>
        <p>&copy; 2026</p>
    </footer>

</body>
</html>
```

`cursos.html`

Vamos agora criar a página `cursos.html` que integra informação dos cursos. Este ficheiro é simples, pois estende o conteúdo da `base.html`, especificando apenas o conteúdo do `{% block content %}{% endblock %}`.

> A página `cursos.html` usa como **layout base** `base.html`. Tal é especificado com a instrução `{% extends "escola/base.html" %}` Neste ficheiro especifica-se o conteúdo do bloco `{% block content %}`.
>
> Utilize sempre elementos HTML5 `section`, `article`, `header` (evite `div`) e siga boas práticas de UI/UX. A estilização será feita posteriormente.

Analise com atenção a forma como se usa em baixo elementos HTML5 para estruturar os conteúdos. É essencial para ficar claro e facilmente estilizável.

```
<!-- templates/escola/cursos.html -->

{% extends "escola/base.html" %}

{% block content %}
```

```
<h2>Cursos</h2>

<section>
    {% for curso in cursos %}
    <article>
    <!-- cada curso é um article,
     com um header com o nome do curso e do professor,
     e uma section com a lista de cursos-->
        <header>
            <h3>{{ curso.nome }}</h3>
            <img src="{{ curso.imagem.url }}" alt="Imagem do logotipo {{ curso.imagem.name }}
            <p>Professor: {{ curso.professor.nome }}</p>
        </header>

        <section>
            <h4>Alunos</h4>
            <ul>
                {% for aluno in curso.alunos.all %}
                    <li>{{ aluno.nome }}</li>
                {% empty %}
                    <li>Sem alunos inscritos</li>
                {% endfor %}
            </ul>
        </section>
    </article>
    {% endfor %}
</section>
{% endblock %}
```

Rota

Cada nova função view & template HTML que cria, deve ter um caminho que permite chamá-la (fazer um *request*). Esse caminho é definido no ficheiro `urls.py`.

Na pasta `escola/`, crie o ficheiro `urls.py`, e defina um caminho (também chamado de rota ou *path*) que despolta a execução da função view correspondente.

```
## escola/urls.py

from django.urls import path
from . import views

urlpatterns = [
    path('cursos/', views.cursos_view, name="cursos"),
]
```

o atributo `name` pode ser usado em links, como está a ser feito em `base.html`, no `nav`.

Visualize a aplicação

Agora pode visualizar a aplicação. Ponha a correr o servidor web de desenvolvimento:

```
python manage.py runserver
```

clique no link que aparece para abrir o browser e complete o URL com o caminho para esta view, `/escola/cursos`; deverá ser capaz de ver a página renderizada em http://127.0.0.1:8000/escola/cursos ↗ .

Mostre ao seu professor.

Melhoria

Vamos definir rotas mais simples para ter um link direto.

```
## project/urls.py

urlpatterns = [
    path("admin/", admin.site.urls),
    path("escola/", include("escola.urls")),
    path("", include("escola.urls")),  #  rota para app escola sem precisar de escrever "esco
]
```

```
## escola/urls.py

from django.urls import path
from . import views

urlpatterns = [
    path('cursos/', views.cursos_view, name="cursos"),
    path('', views.cursos_view),   #  rota que abre diretamente a página dos cursos
]
```

ⓘ com esta alteração, ao clicar em http://127.0.0.1:8000/ ↗ abrirá diretamente a página das escolas

Estilização de base.html

ⓘ Podemos estilizar o ficheiro `base.html`, afectando todas as páginas da aplicação. É uma das vantagens. Podemos usar cores e dispor elementos usando CSS.

Estilize agora o seu template base.html. Crie um ficheiro de estilos na pasta `escola/static/escola/styles.css` com o conteúdo em baixo. Pode alterar a seu gosto.

```
/* escola/static/escola/styles.css   */

* {
    box-sizing: border-box;
    margin: 0;
    padding: 0;
}

/* BODY */
body {
    font-family: "Inter", "Segoe UI", system-ui, -apple-system, sans-serif;
    background: #e6ebf2;
    color: #111827;
    line-height: 1.5;
}

/* HEADER PRINCIPAL DO WEBSITE (de base.html) */
body > header {
    background: #1f2937;
    color: white;
    padding: 20px 40px;
}

body > header h1 {
    font-size: 26px;
}

body > header nav {
    margin-top: 10px;
}

body > header nav a {
    color: #d1d5db;
    margin-right: 20px;
    text-decoration: none;
    font-weight: 500;
}

body > header nav a:hover {
    color: white;
}

/* MAIN */
main {
    max-width: 1100px;
    margin: 40px auto;
    padding: 0 20px;
}

/* TÍTULO DA PÁGINA */
h2 {
    margin-bottom: 20px;
    font-size: 26px;
}

/* GRID DE CURSOS */
section > article {
    background: white;
    padding: 20px;
    border-radius: 16px;
    border: 1px solid #d1d5db;
    box-shadow: 0 10px 25px rgba(0,0,0,0.08);
    transition: all 0.25s ease;
    margin-bottom: 20px;
}

section > article:hover {
    transform: translateY(-6px);
    box-shadow: 0 16px 35px rgba(0,0,0,0.18);
}

/* HEADER DO CARD */
article > header {
    margin-bottom: 12px;
}

article > header > h3 {
    font-size: 20px;
    font-weight: 600;
    margin-bottom: 4px;
}

article > header > p {
    font-size: 14px;
    color: #374151;
}

/* SEÇÃO DO CARD */
article > section {
    margin-top: 10px;
}

article > section > h4 {
    font-size: 11px;
    text-transform: uppercase;
    color: #6b7280;
    margin-bottom: 8px;
    letter-spacing: 0.05em;
}

/* LISTA DE ELEMENTOS do CARD */
article > section > ul {
    list-style: none;
    display: flex;
    flex-wrap: wrap;
    gap: 8px;
}

/* ITEM */
article > section > ul > li {
    background: #2563eb;
    color: white;
    padding: 6px 10px;
    border-radius: 999px;
    font-size: 13px;
    font-weight: 500;
}

/* FOOTER */
footer {
    text-align: center;
    padding: 25px;
    background: #1f2937;
    color: #9ca3af;
    margin-top: 40px;
}
```

Agora insira em `base.html`, no elemento `head`, um link para esse ficheiro, de modo a que sejam aplicados os estilos. Use a sintaxe do Django, que requer `{% load static %}` (indica o caminho completo para a pasta static) e que indique o caminho com o elemento link `<link rel="stylesheet" href="{% static 'escola/style.css' %}">`

```
<!-- base.html -->
<!DOCTYPE html>
<html lang="pt">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Escola</title>

    <!-- link para ficheiro styles.css com regras de estilização -->
    {% load static %}
    <link rel="stylesheet" href="{% static 'escola/styles.css' %}">

    <!-- ... -->
```

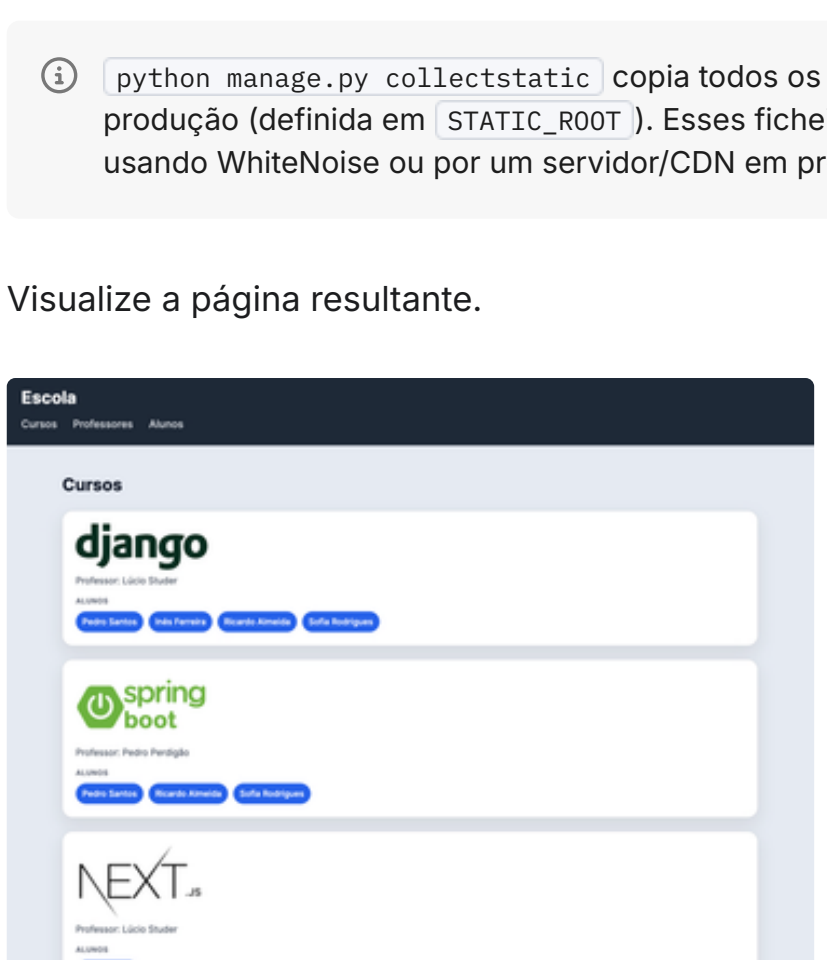
ⓘ `{% load static %}` ativa o uso da tag `static` em `{% static 'escola/style.css' %}` para gerar URLs de ficheiros estáticos.

Agora execute o comando

```
python manage.py collectstatic
```

ⓘ `python manage.py collectstatic` copia todos os ficheiros estáticos das aplicações para uma pasta única de produção (definida em `STATIC_ROOT`). Esses ficheiros podem depois ser servidos diretamente pelo Django usando WhiteNoise ou por um servidor/CDN em produção. No nosso caso, será pelo WhiteNoise.

Visualize a página resultante.



Escola

Cursos

Professor: Lúcio Studer

ALUNOS

Pedro Santos, Inês Ferreira, Ricardo Almeida, Sofia Rodrigues

Professor: Pedro Perdigão

ALUNOS

Pedro Santos, Ricardo Almeida, Sofia Rodrigues

Professor: Lúcio Studer

ALUNOS

Inês Ferreira

Professor: Luís Sousa

ALUNOS

Inês Ferreira, Sofia Rodrigues

© 2026

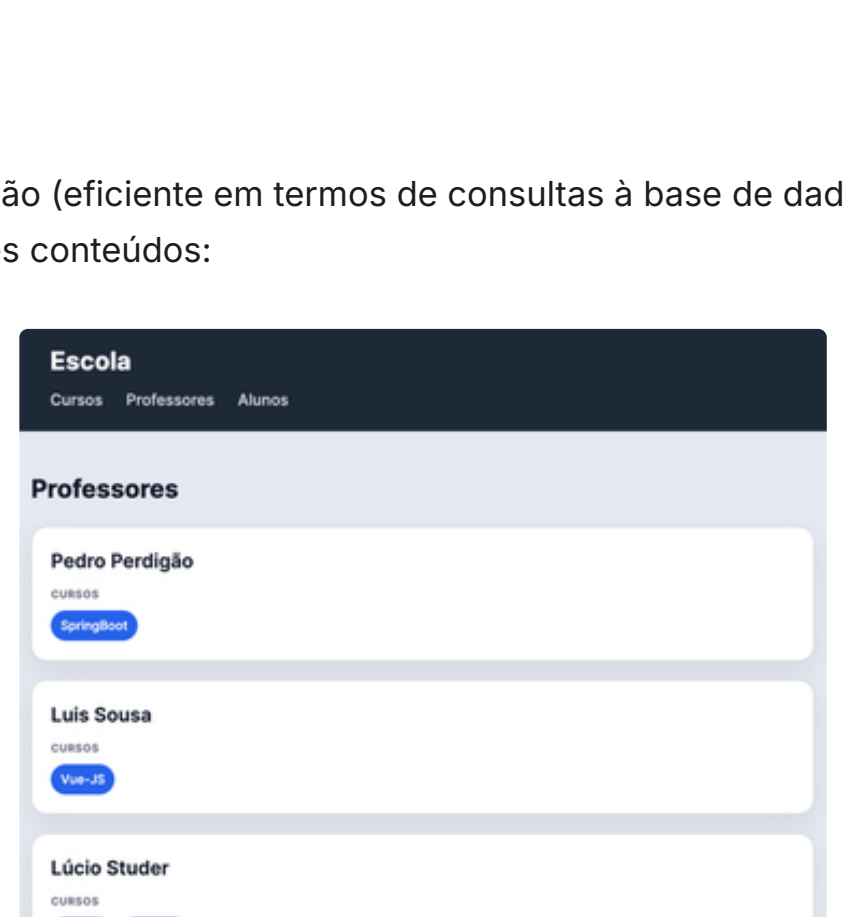
Crie visualizações de Professores e Alunos

ⓘ Irá agora repetir os mesmos passos, mas para outras duas páginas. Assim sistematiza e perceberá que é um processo simples. Para cada nova página devemos:

1. criar em `escola/views.py` uma nova função que vá buscar os conteúdos à base de dados e os envie para serem renderizados num template HTML
2. criar em `templates/escola` um ficheiro template HTML que renderiza (insere) os conteúdos da bae de dados no HTML. Use os mesmos elementos HTML5 (`section` , `h2` , `header` , `h2` , `article` , ...) e da mesma forma, para aproveitar a estilização que é comum.
3. definir em `escola/urls.py` um novo caminho que permite chamá-la (fazer um *request*).

Professores

Implemente uma visualização (eficiente em termos de consultas à base de dados) que permita a apresentação dos seguintes conteúdos:



Escola

Professores

Pedro Perdigão

CURSOS

SpringBoot

Luis Sousa

CURSOS

Vue-JS

Lúcio Studer

CURSOS

Django, Next-JS

Ricardo Santos

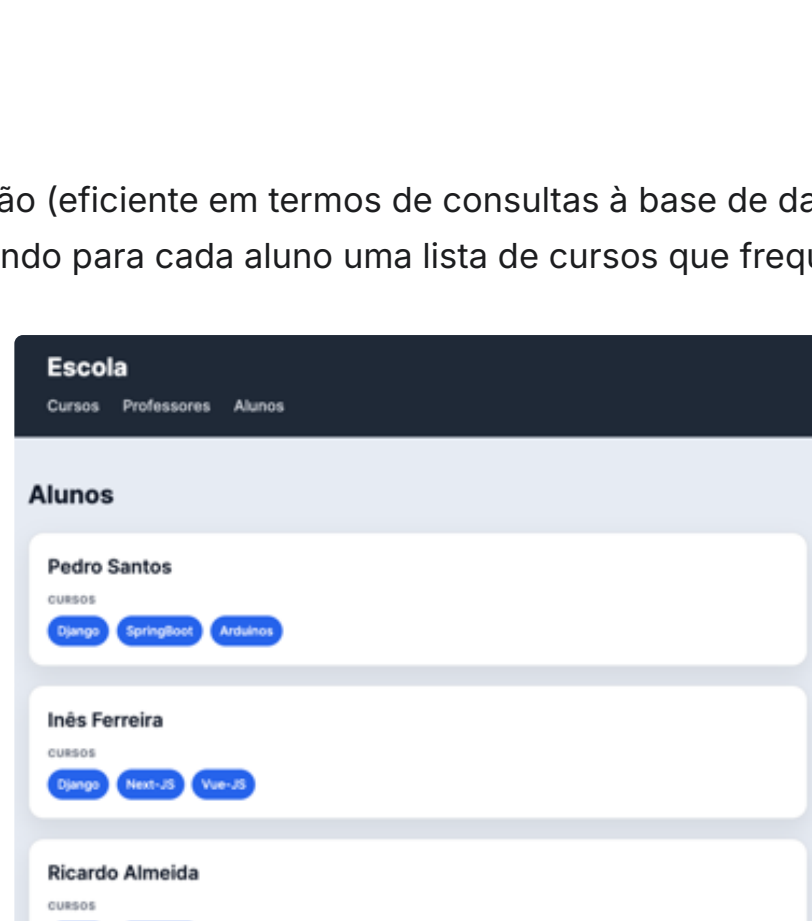
CURSOS

Arduinos

© 2026

Alunos

Implemente uma visualização (eficiente em termos de consultas à base de dados) que permita visualizar a lista de alunos, apresentando para cada aluno uma lista de cursos que frequenta.



Escola

Alunos

Pedro Santos

CURSOS

Django, SpringBoot, Arduinos

Inês Ferreira

CURSOS

Django, Next-JS, Vue-JS

Ricardo Almeida

CURSOS

Django, SpringBoot

Sofia Rodrigues

CURSOS

Django, SpringBoot, Vue-JS, Arduinos

© 2026

Visualização de um curso

Na página de alunos, podemos querer colocar um link em cada curso, que mostre numa página informação desse curso.

```
<!-- templates/escola/alunos.html -->

{% extends "escola/base.html" %}

{% block content %}
<h2>Alunos</h2>

<section>
    {% for aluno in alunos %}
    <article>
        <header>
            <h3>{{ aluno }}</h3>
        </header>

        <section>
            <h4>Cursos</h4>
            <ul>
                {% for curso in aluno.cursos.all %}
                    <li>
                        <a href="{% url 'curso' curso.id %}">
                        {{ curso }}
                        </a>
                    </li>
                {% endfor %}
            </ul>
        </section>
    </article>
    {% endfor %}
</section>
{% endblock %}
```

Agora criamos uma nova rota para um curso:

```
urlpatterns = [
    path('cursos/', views.cursos_view, name="cursos"),
    path('curso/<int:id>', views.curso_view, name="curso"),
]
```

Depois criamos uma view para mostrar a pagina de um curso

```
## ficheiro escola/views.py

from django.shortcuts import render
from .models import Curso

def curso_view(request, id):
    curso=Curso.objects.get(id=id)
    return render(request, 'escola/curso.html', {'curso': curso})
```

E finalmente criamos a página HTML do curso

```
<!-- templates/escola/curso.html -->

{% extends "escola/base.html" %}

{% block content %}
    <h2>Curso {{ curso }} </h2>
    <h3>{{ curso.nome }}</h3>
    <img src="{{ curso.imagem.url }}" alt="Imagem do logotipo {{ curso.imagem.name }}">
    <p>Professor: {{ curso.professor.nome }}</p>

    <section>
        <h4>Alunos</h4>
        <ul>
            {% for aluno in curso.alunos.all %}
                <li>{{ aluno.nome }}</li>
            {% empty %}
                <li>Sem alunos inscritos</li>
            {% endfor %}
        </ul>
    </section>
{% endblock %}
```

Commit

Faça commit para o GitHub das alterações feitas.

Portfolio

Mostre ao seu professor a modelação que fez para o portfolio.

Para o seu projeto em Django (portfólio), deve implementar **interfaces de consulta (listagem)** para os dados de cada modelo criado no sistema (por exemplo: tecnologias, disciplinas, TFCs, projetos, entre outros).

1. Para tal, deve editar o ficheiro projects/urls.py de modo a incluir um caminho para portfolio.
2. Para cada modelo, deverá depois desenvolver:

- uma **view**
- um **template**
- a respetiva **URL**

Commit

Faça commit das alterações para o GitHub.

Only this page
All pages
Powered by GitBook